

LAB1 · VS Code 환경설정

빈 템플릿에서 직접 작성하고 Icarus로 시뮬레이션한다.

Git · Python · Icarus Verilog · VS Code · VaporView

작성일 2026. 09. 10. · 이해리

22개 실습 전체 목차

설치와 실습 목차

1. 편집기·Git·Python·Icarus 설치
도구 설치와 경로 확인

2. 빈 프로젝트 시작
태그 clone·새 창·추천 확장

3. 직접 작성·실행·파형
세 작업과 문제 해결

첫 논리 게이트의 전체 코드와 클릭 실습

도구의 역할

1. VS Code: 파일 편집과 작업 실행.
2. Git: 템플릿을 받고 자신의 변경을 기록.
3. Python 3.10 이상: tools/lab1.py 실행.
4. Icarus Verilog: iverilog로 컴파일, vvp로 시뮬레이션.
5. slang: 편집 중 진단. VaporView: 생성된 VCD 파형 표시.
6. Vivado: 이후 같은 소스로 시뮬레이션·합성·구현. CLI 과정은 해당 도구를 사용한다.

Windows · VS Code와 Git

1. 아래 공식 VS Code 설치 안내에서 Windows User Setup을 받는다.
2. 설치 화면의 PATH 추가 항목을 확인하고 설치를 마친다.
3. Git for Windows도 설치하고 명령줄에서 Git을 사용할 수 있게 한다.
4. 설치 후 기존 VS Code와 터미널을 닫고 새로 연다.

VS Code 공식 설치 Git 공식 설치

Windows · Python 설치

1. Python 공식 Windows 설치 프로그램을 실행한다.
2. Add python.exe to PATH를 선택하고 Install Now를 누른다.
3. 설치 후 VS Code와 PowerShell을 다시 연다.
4. 아래 명령이 실제 Python 버전을 출력하는지 확인한다.

```
python --version
```

제작 PC에서는 Python 3.12.10으로 실습했다. 3.10 이상이 필요하다.
Python 공식 Windows 설치 안내

Windows · Icarus 설치

1. MSYS2 공식 설치 안내에 따라 설치·초기 업데이트를 마친다.
2. 시작 메뉴의 MSYS2 MINGW64 터미널을 연다.
3. 아래 명령을 입력하고 패키지 설치를 완료한다.

```
pacman -S mingw-w64-x86_64-iverilog
```

기본 설치 경로는 C:/msys64/mingw64/bin이다. iverilog.exe와 vvp.exe를 함께 제공한다.

MSYS2 설치 Icarus 패키지·설치 명령

Windows · Icarus를 PATH에 추가

1. 시작 메뉴에서 계정의 환경 변수 편집을 검색해 연다.
2. 사용자 변수의 Path 선택 → 편집 → 새로 만들기.
3. C:/msys64/mingw64/bin을 추가하고 열린 창의 확인을 누른다.
4. 설치 위치를 바꿨다면 자신의 mingw64/bin 경로를 넣는다.
5. VS Code를 모든 창까지 종료하고 다시 연다.
6. Terminal → New Terminal에서 다음 장의 버전을 확인한다.

Windows PowerShell · 버전 확인

```
git --version  
python --version  
iverilog -V  
vvp -V  
Get-Command python, iverilog, vvp
```

이름을 찾을 수 없으면 설치 위치와 PATH를 확인한다.

Python 대신 Store가 열리거나 버전이 없으면 실제 Python 설치와 호출 경로를 확인한다.

macOS · Linux · WSL 에서 시작할 때

Git · Python 3 · Icarus를 해당 운영체제에 설치한다. Python 명령은 python3를 사용한다.

WSL은 VS Code의 WSL 확장으로 Linux 폴더를 열고 확장도 Install in WSL로 설치한다.

Windows와 WSL의 도구 · 경로를 섞지 않는다. 통합 CLI의 ARM64 배포와 WSL 실행 범위는 별도 교안을 따른다.

```
git --version
python3 --version
iverilog -V
vvp -V
```

통합 CLI 환경 안내

v2.0.0 템플릿을 새 이름으로 받기

Windows PowerShell에서 실습 폴더를 둘 위치로 이동한 뒤 실행한다. 줄 끝의 백틱은 다음 줄로 이어 준다.

```
git clone --branch v2.0.0 `
https://github.com/Glaysia/fpga-lab-template.git `
lab1_01_logic_gates
cd lab1_01_logic_gates
git switch -c main
```

다음 회로에서는 마지막 폴더 이름을 바꾼다. 모든 회로가 같은 단일 프로젝트 템플릿에서 출발한다.

macOS · Linux · 같은 태그 clone

이 장은 Bash · Zsh 용이다. 마지막 목적지 이름을 회로마다 바꾼다.

```
git clone --branch v2.0.0 \  
https://github.com/Glaysia/fpga-lab-template.git \  
lab1_01_logic_gates  
cd lab1_01_logic_gates  
git switch -c main
```

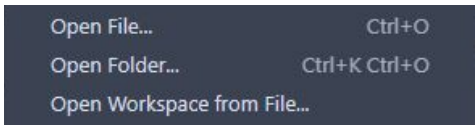
코드 작성은 해당 회로 PDF의 RTL · TB · XDC 전체 이미지를 따른다.

File → New Window

VS Code 상단 File → New Window를 누른다. Ctrl+Shift+N도 같다.



새 창에서 File → Open Workspace from File...을 누른다.



LAB1.code-workspace 열기

1. 방금 clone한 폴더로 이동한다.
2. LAB1.code-workspace를 선택하고 Open을 누른다.
3. Explorer의 PROJECT 아래에 src.sim.constraints.tools가 보이는지 확인한다.
4. RTL.TB.XDC는 주석뿐인 시작 파일이다. 회로 PDF를 보며 작성한다.
5. Restricted Mode라면 출처와 실행할 작업을 읽고 해당 프로젝트의 신뢰 여부를 직접 결정한다.

추천 확장 세 개 설치

1. 왼쪽 Extensions 아이콘 클릭 → 검색창에 @recommended 입력.
2. slang → Install: Verilog/SystemVerilog 편집 진단.
3. VaporView → Install: VCD 파형 보기.
4. vscode-pdf → Install: PDF 교안을 편집기 옆에서 보기.
5. 이미 설치되어도 Disabled 상태이면 해당 확장의 Enable (Workspace)을 선택한다.

확장 설치와 Python·Icarus 설치는 각각 필요하다.

빈 파일을 내 회로 파일로 바꾸기

1. Explorer에서 src/design.v 우클릭 → Rename. 회로 PDF의 RTL 파일명으로 변경한다.
2. sim/tb_design.v와 constraints/pins.xdc도 같은 방법으로 변경한다.
3. 각 파일을 두 번 눌러 열고 전체 코드 이미지를 보며 직접 입력한다.
4. RTL이 여러 개라면 src 우클릭 → New File로 각각 만든다.
5. File → Save All. 파일명과 module 이름을 구분한다.

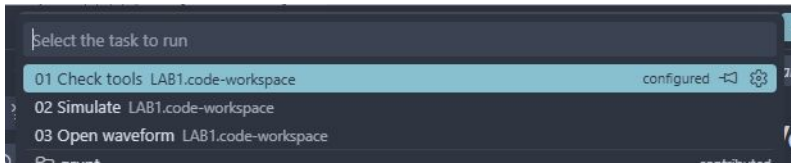
simulation.json 수정

1. Explorer에서 simulation.json을 두 번 누른다.
2. sources 배열에 자신이 만든 src/... 파일들을 모두 등록한다.
3. testbench에는 sim/... 파일 경로를 적는다.
4. simulation_top에는 테스트벤치 module 이름을 적는다.
5. 경로의 큰따옴표·쉼표·대괄호를 확인하고 Save All.

첫 회로의 전체 JSON·RTL·TB·XDC 화면

세 작업을 순서대로 실행

Terminal → Run Task...을 누른다. 메뉴가 숨겨졌으면 상단 ...을 확인한다.



1. 01 Check tools: Python·Icarus 실행을 확인한다.
2. 02 Simulate: 작성한 RTL·TB를 컴파일하고 실행한다.
3. 03 Open waveform: 생성된 wave.vcd를 연다.

Vivado 실행은 이 세 작업에 포함되지 않는다. 해당 실습의 Vivado GUI 절차를 뒤에서 진행한다.

PASS·검사 수·시간 읽기

SIMULATED는 실행기의 완료 상태다. TB의 LAB1_PASS와 검사 수까지 확인한다.

첫 논리 게이트는 4개 검사, 40ns 종료다. 회로마다 검사 수·종료 시각은 다르므로 자신의 PDF와 대조한다.

build/sim/result.json과 실행별 simulation.log, wave.vcd를 보관한다.

실패 시 FAIL 문구의 입력·예상값·실제값을 읽고 코드 수정 → Save All → 02 Simulate로 다시 실행한다.

VaporView 파형 열기

1. 03 Open waveform을 실행한다.
2. 텍스트로 열리면 VCD 탭 우클릭 → Reopen Editor With... → VaporView.
3. Netlist에서 테스트벤치를 펼치고 입력·출력 신호를 추가한다.
4. Zoom to Fit으로 전체 시간을 본다.
5. 구간 중간에 커서를 두고 입력과 출력 값을 진리표와 비교한다.
6. 파형이 비어 있으면 신호 추가 여부와 이번 실행의 VCD인지 확인한다.

실행이 안 될 때 확인할 순서

1. 작업 메뉴가 없다: 올바른 LAB1.code-workspace를 열었는지 확인.
2. Python·iverilog를 못 찾는다: Get-Command 결과, PATH, VS Code 재시작 확인.
3. 컴파일 실패: 로그의 첫 오류, 파일명·세미콜론·괄호 확인.
4. top을 찾지 못한다: simulation_top과 TB module 선언 비교.
5. FAIL 또는 timeout: 입력·기대값·종료 조건과 watchdog 확인.
6. 파형 편집기가 없다: VaporView 설치·Enable (Workspace) 확인.

설치 완료 후 첫 실습으로

1. 세 도구의 버전과 호출 경로를 확인했다.
2. 빈 템플릿을 v2.0.0으로 받고 workspace 하나를 열었다.
3. 추천 확장 세 개를 설치했다.
4. 회로 PDF를 보며 RTL·TB·JSON·XDC를 직접 작성한다.
5. 실험 전에는 VS Code 결과, 실험 후에는 구현·실측·사진·영상·GitHub 링크를 정리한다.

01 논리 게이트 시작 22개 전체 목차